

# SLAC: Access-Driven CPU-to-GPU Side-channel Attacks via System-Level Cache on Apple Silicon

Tianhong Xu  
Northeastern University  
Boston, USA  
xu.tianh@northeastern.edu

Saion K. Roy  
Northeastern University  
Boston, USA  
sai.roy@northeastern.edu

Ruyi Ding  
Louisiana State University  
Baton Rouge, USA  
ruyiding@lsu.edu

Aidong Adam Ding  
Northeastern University  
Boston, USA  
a.ding@northeastern.edu

Yunsi Fei  
Northeastern University  
Boston, USA  
y.fei@northeastern.edu

## Abstract

Modern heterogeneous System-on-Chip designs integrate CPU cores and a GPU that share a last-level cache (LLC) or system-level cache (SLC). This sharing exposes a new cross-domain attack surface, and existing attacks on integrated platforms either exploit coarse-grained cache-occupancy contention or require the adversary to co-reside on the GPU with the victim to obtain accurate timing measurements. In this work, we target Apple Silicon heterogeneous SoCs and discover that GPU memory accesses leave set-level footprints in the shared SLC, observable to an unprivileged CPU process. This keen observation enables the first fine-grained, access-driven, Prime+Probe-style CPU-to-GPU cache side-channel attacks against GPU workloads. We first reverse-engineer the Apple M1 SLC set-indexing functions and the interactions between local private caches and the SLC. Building on these findings, we construct the CPrime+CProbe SLC side-channel technique, which monitors GPU victim activity from the CPU at cache-set granularity. We then introduce an accelerated variant, GPrime+CProbe, in which an adversary leverages the GPU for faster SLC priming, yielding a 6.4× increase in the covert-channel throughput. Lastly, we demonstrate two end-to-end privacy attacks using the new side-channels: a graph-edge reconstruction attack on Graph Neural Networks (GNNs) that achieves 90% edge accuracy across five datasets, and an LLM privacy attack that recovers input keywords with up to 94.8% accuracy and model responses with up to 88.9% accuracy across TinyLlama and GPT-2 Medium models. Our results reveal a new class of microarchitectural vulnerabilities in Apple Silicon and call for secure system cache designs for heterogeneous SoCs.

## CCS Concepts

• Security and privacy → Side-channel analysis and counter-measures.

## Keywords

Apple M1, Cache Attacks, Heterogeneous Systems, ML Security

## ACM Reference Format:

Tianhong Xu, Saion K. Roy, Ruyi Ding, Aidong Adam Ding, and Yunsi Fei. 2026. SLAC: Access-Driven CPU-to-GPU Side-channel Attacks via System-Level Cache on Apple Silicon. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3830454.3846599>

## 1 Introduction

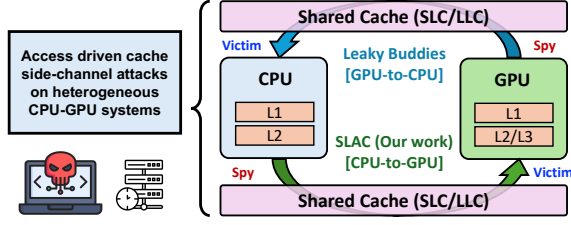
Modern GPUs are used extensively for privacy-sensitive workloads such as machine learning (ML) inference and cryptographic operations, making information leakage from these computations a pressing security concern. Heterogeneous computing systems integrate CPU and GPU on the same system-on-chip (SoC), and their shared microarchitectural resources create a new cross-domain path to confidentiality and privacy breaches of GPU workloads. Realizing such microarchitectural attacks, however, is nontrivial and highly platform-dependent: *the success hinges on which microarchitectural resources are shared between CPU and GPU, how those resources are accessed or contended by the two sides, and whether GPU activities can be reliably observed from the CPU side.*

We classify cross CPU-GPU side-channel threats by the respective execution domains of the adversary and the victim. Leaky Buddies [6] demonstrates a proof-of-concept GPU-to-CPU side-channel on Intel integrated platforms, in which a GPU-resident adversary monitors the state of the shared Last-Level Cache (LLC) as affected by CPU activities. The more pressing threat model for sensitive GPU workloads is the opposite CPU-to-GPU direction, in which a CPU-side adversary attacks a GPU victim. Existing attacks in this direction, however, remain limited to coarse-grained cache-occupancy attacks on ARM [5] and Apple Silicon [32], which observe only aggregate cache usage rather than individual cache sets or lines. Other works also demonstrate bi-directional covert channels through the shared LLC on Intel integrated platforms [6, 13]. To the best of our knowledge, *no prior work has demonstrated a set-level, access-driven CPU-to-GPU side-channel via a shared cache.* Fig. 1 summarizes the status quo.

Establishing a fine-grained Prime+Probe side channel from CPU-to-GPU is substantially harder than in earlier results, because the CPU-side adversary must prime the shared cache and reliably observe its state changes at cache-set granularity due to GPU victim activity. This requires a deep understanding of *when, where, and*



This work is licensed under a Creative Commons Attribution 4.0 International License. *CCS '26, The Hague, Netherlands*  
© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2871-6/2026/11  
<https://doi.org/10.1145/3830454.3846599>



**Figure 1: Landscape of access-driven cache side-channel attacks for heterogeneous CPU-GPU architectures.**

how the GPU victim’s data accesses affect the shared cache state. Among the three cache inclusion policies [26] that govern shared-cache residency with respect to a private cache, namely exclusive, inclusive, and non-inclusive non-exclusive (NINE), only with the latter two can a victim data access survive the private caches and affect the shared cache state. Under the exclusive policy, a data access results in cache line residence in the private caches, but absent from the shared cache, making the shared cache state non-indicative of data accesses. Under the inclusive policy, every line in the victim’s private cache is also always held in the shared cache, while under NINE, the cache line resides in the shared cache conditionally. Only in these two cases can the shared cache reveal victim data access or no-access to a spy process.

We target Apple’s M-series SoCs, which have become an attractive platform for privacy-sensitive local AI workloads thanks to their high-performance, energy-efficient GPU acceleration. On Apple Silicon, the SLC is exclusive of the CPU caches and holds a NINE relationship with the GPU caches [32]. Victim information can leak through the SLC via the GPU-side NINE relationship, whereas the CPU-side exclusive relationship governs how the adversary sets up the shared cache and monitors its state changes. A CPU-to-GPU attack is therefore feasible with the specific cache inclusiveness.

The challenge, however, lies in how the adversary primes the shared cache and monitors its state. Because the SLC is exclusive of the CPU caches, a line cannot reside in both the CPU’s private caches and the SLC. A CPU adversary has to saturate L2 cache sets first and rely on evictions due to capacity conflicts to spill older cache lines into the SLC. This two-step priming process is fundamentally different from the counterpart used in the prior GPU-to-CPU side-channel on Intel heterogeneous platforms [6]. There, the GPU directly primes the LLC via data accesses. However, due to the NINE relationship between the LLC and GPU private caches, primed cache lines may also have a copy in the GPU’s private caches. Those copies must be cleared from the GPU’s private caches; otherwise, they will interfere with the subsequent probing (access timing measurement) phase, leading to inaccurate observations of victim data activity. In addition to these differences in cache hierarchy and inclusiveness, public knowledge about Apple Silicon’s SLC is much less than that of Intel LLC.

For cache attacks on Apple Silicon, the prior work [32] only demonstrates coarse-grained SLC-occupancy attacks, which cannot resolve individual cache lines/sets and therefore cannot recover fine-grained secrets such as ML model inputs or cryptographic keys. Achieving set-level observability further requires architectural knowledge of the Apple SLC that is not publicly available.

To close this gap, we present **SLAC**, the first access-driven Prime+Probe SLC side-channel attack on Apple M-series SoCs targeting sensitive GPU workloads. We first develop a reverse-engineering methodology to recover the SLC’s set-indexing functions and characterize the interactions between the CPU/GPU private caches and the shared SLC. Building on these findings, we construct **CPRIME+CPROBE**, our core CPU-to-GPU side-channel technique, which executes both SLC priming and probing on the CPU and requires no GPU access from the adversary. A covert channel built on **CPRIME+CPROBE** achieves a throughput of 62.5 Kbps. We further develop an accelerated extension, **GPrime+ CProbe**, that primes the SLC from the GPU while retaining the CPU-side probe. GPU parallelism makes priming about 6.4× faster than **CPRIME**, and the resulting covert channel reaches 400 Kbps. We then demonstrate two end-to-end privacy attacks built on these techniques, namely a graph-structure reconstruction attack on Graph Neural Networks (GNNs) that recovers over 90% of edges across five datasets, and an input and output retrieval attack on large language models (LLMs) that recovers input keywords with up to 94.8% accuracy and output tokens with up to 88.9% accuracy on TinyLlama and GPT-2 Medium.

The contributions of this work include:

- (1) We reverse-engineer the Apple M1’s SLC set-indexing functions and characterize the interactions between private caches and SLC that underpin CPU-observability of GPU memory accesses.
- (2) We propose the first access-driven CPU-to-GPU side-channel on Apple Silicon, **CPrime+CPProbe**, and an accelerated variant, **GPrime+CPProbe**, which yields a 6.4× improvement of the covert-channel throughput.
- (3) We demonstrate two end-to-end privacy attacks on GPU workloads: GNN edge recovery that reconstructs over 90% of edges across five datasets, and LLM input/output recovery that reaches up to 94.8% and 88.9% accuracy on TinyLlama and GPT-2 Medium.

Our results reveal that cache sharing between CPUs and GPUs in increasingly popular heterogeneous systems poses serious security vulnerabilities and demands careful architectural redesign to ensure the security and privacy of critical GPU workloads.

The rest of the paper is organized as follows. Section 2 reviews background and prior relevant work. Section 3 presents our reverse engineering of the Apple M1 SLC. Section 4 describes our CPU-to-GPU Prime+Probe side-channel designs and their covert-channel evaluation. Section 5 demonstrates end-to-end privacy attacks on GPU workloads. Section 6 concludes the paper with a discussion of future work.

## 2 Background and Related Work

This section provides background on cache side-channel attacks and the Apple M-series SoC architecture. We also position our work with respect to prior research on cross CPU-GPU attacks and reverse-engineering of cache set-indexing.

### 2.1 Cache Side-Channel Attacks

Cache is the most common shared microarchitecture exploited in side-channel attacks, where the timing difference between cache hit and miss is used to identify cache state change. *Cache occupancy*

attacks monitor the overall usage or contention level of the shared cache rather than targeting individual cache sets. This side-channel is easy to deploy, robust to noise, and requires only that the spy be familiar with architectural basics, such as the cache inclusion policy. However, it only provides coarse-grained information leakage, and is typically used for website fingerprinting [5, 23, 24, 32] and data volume-based pattern recognition [8, 14, 32].

The other type, *access-driven side-channel*, offers significantly finer spatial and temporal granularity. It monitors accesses of specific cache sets or lines by the victim. Among them, Prime+Probe is the most widely applicable technique, as it does not require shared memory or special instructions (both are required for Flush+Reload side-channels). The salient steps of setting up Prime+Probe side-channels are:

- **Prime phase:** The adversary sets the shared cache to a certain state by filling one chosen cache set with its own data. Specifically, an eviction set, which includes a group of adversary data addresses, should be constructed beforehand to be used for priming the cache. The eviction set and the victim secret contend on a cache set, i.e., both map to the cache set through indexing functions over their memory addresses.
- **Victim execution:** The victim executes its program that may access the secret, therefore causing one replacement on the cache set and evicting one cache line of the adversary.
- **Probe phase:** The attacker re-accesses the eviction set and measures the access latency. A higher latency indicates an eviction by the victim, and hence infers an access of the target victim memory address; while a low latency corresponds to victim no-access.

Prime+Probe attacks have been successfully deployed in many scenarios on various CPU architectures [7, 16, 19, 23, 29, 33]. However, no prior work has realized a Prime+Probe CPU-to-GPU cache side-channel on integrated SoCs.

## 2.2 Apple M-series SoCs

Apple M-series Systems-on-Chip (SoCs), including the M1, M2, and their subsequent variants, are built with the ARMv8-A Instruction Set Architecture (ISA). Apple M-series SoCs adopt **Unified Memory Architecture (UMA)**, in which the CPU, GPU, and other accelerators access a common pool of physical memory. Apple’s Metal framework, the graphics and compute shader API, provides a unified programming interface for heterogeneous CPU–GPU execution, enabling shared memory to be accessible from both the CPU host and the integrated GPU. This design avoids data copies between host and device memories, thereby improving workload performance.

Figure 2 depicts the cache hierarchy of Apple M1 and its respective sizes. Specifically, M1 has four high-performance Firestorm cores (p-cores) and four energy-efficient Icestorm cores (e-cores), forming two CPU clusters with core-specific L1 caches and cluster-specific L2 caches. The SLC is shared across the two clusters and the GPU, playing a central role in mediating data access in the UMA design. The cache line size is 128 bytes for all the caches. In practice, macOS schedules active processes onto the Firestorm cluster while reserving Icestorm for low-priority background work. We therefore

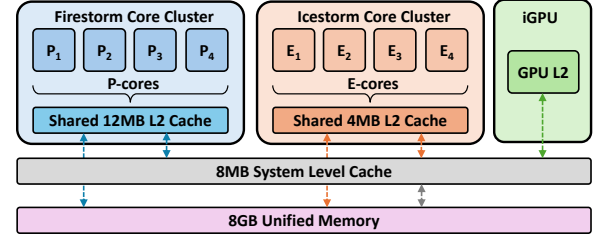


Figure 2: Cache structure of Apple M1.

focus all subsequent experiments on the Firestorm cluster, with CPU being the P-cores and CPU L2 their 12MB L2 cache.

**2.2.1 Prior Reverse Engineering and Cache Attacks on Apple Silicon.** Prior work [32] characterized several key properties of the Apple M1 SLC and demonstrated occupancy-based cache attacks on it.

**Cache Residency Policy:** The SLC shows asymmetric cache residency behavior across the CPU and GPU domains. With respect to CPU private caches, the SLC is exclusive: for CPU memory accesses, cache lines are initially kept in the CPU’s private caches, and are only ejected to the SLC upon eviction from the CPU L2 cache. In contrast, for GPU memory accesses, corresponding cache lines are directly loaded into the SLC, with a NINE relationship relative to the GPU private caches. With this insight, an access-driven CPU-to-GPU side-channel is feasible.

**Set Indexing:** The Apple M1 SLC employs a non-traditional set-indexing scheme that is fundamentally different from CPU local cache indexing. The set index does not use the lowest 13 bits of the physical address, resulting in a sparse, less intuitive mapping between physical addresses and SLC cache sets and complicating eviction-set construction. There is no prior public knowledge of Apple Silicon SLC set indexing functions.

**Occupancy-based Cache Attacks:** The prior work [32] only demonstrated cache-occupancy attacks on Apple M1, including website fingerprinting and screen capture via aggregate SLC usage. Fine-grained, access-driven CPU-to-GPU leakage on Apple Silicon, however, remains unexplored and is the focus of this work.

## 2.3 Related Work on Cross CPU-GPU Platforms

Cross CPU-GPU Prime+Probe cache side-channels on integrated SoCs have so far been studied only on Intel platforms, in Leaky Buddies [6] with proof-of-concept preliminary results. Cache set-indexing functions on Intel platforms have been extensively reverse-engineered in prior work [9, 10, 17, 28, 35], which showed that Intel’s LLC set indexing uses XOR hash functions over 6 bits or more of the physical address. Recovering these functions has enabled practical cache attacks on X86 systems. Beyond Intel, however, set-indexing for other heterogeneous CPU-GPU systems remains far less understood, particularly on platforms where the shared last-level cache is not inclusive of the CPU private caches. Recent work on AMD processors [30] only demonstrated cross-core side-channel leakage. On Apple’s M-series SoCs, prior work [32] only coarsely characterized the SLC indexing without resolving the concrete hash functions. We close this gap by reverse-engineering the SLC’s set-indexing functions in Section 3 and then building our side-channel techniques in Section 4.

### 3 Reverse Engineering Apple M1's SLC Set Indexing Functions

This section presents our reverse engineering of the Apple M1's SLC set indexing functions, which are a prerequisite for constructing the access-driven SLC side-channel, as targeting a specific cache-set depends on knowing how physical addresses map to sets. This reverse engineering process, however, is non-trivial for two reasons. **Challenge 1: CPU-exclusive SLC residency.** The SLC is exclusive with respect to the CPU private caches, which means that CPU-side data accesses cannot reliably populate or locate SLC cache lines. Standard CPU-based eviction-set construction, therefore, does not apply, and set congruency cannot be inferred from the timing of CPU data access.

**Challenge 2: Fully hashed set indexing.** As noted in Section 2.2, the SLC does not draw its set index from contiguous low-order physical address bits directly. Each cache index bit is derived from a hash function over multiple address bits. This mapping expands the candidate space for eviction-set construction and makes traditional iterative pruning unstable under realistic measurement noise.

We overcome these two challenges in our reverse engineering. Section 3.1 examines how *shared data* is handled between the CPU and GPU, and shows that GPU data accesses can place cache lines into the SLC while the CPU observes their residency, resolving Challenge 1. Building on CPU-side timing measurement, Section 3.2 introduces an eviction-set construction method that replaces noise-sensitive iterative pruning with collision-profile clustering, thereby addressing Challenge 2.

#### 3.1 SLC Behavior in CPU-GPU Data Sharing

We closely track how CPU-GPU *shared data* moves through the system cache hierarchy, between the SLC and CPU private caches. We classify shared data into three representative scenarios based on its initial residency state before sharing:

- ① **CPU-private (L2-resident) to shared.** Data initially resides in the CPU's private L2 cache and is not present in the SLC due to the L2 cache's exclusivity. The GPU subsequently accesses this data block.
- ② **CPU-private (SLC-resident) to shared.** Data was previously evicted from the CPU's L2 cache into the SLC. The GPU then accesses this SLC-resident data.
- ③ **GPU-accessed to shared.** Data is initially accessed by the GPU and resides in the SLC, and the CPU accesses it thereafter.

These three scenarios capture all possible transitions of cached data from a private state to a shared state. For each scenario, we use subsequent CPU-side access and measure its latency to distinguish among CPU local cache hits (L1 or L2), SLC hits, and SLC misses, thereby tracking the precise residence at specific cache levels. Figure 3 summarizes the results for the three scenarios. Note that we focus on the SLC and CPU cache states rather than the GPU cache state, due to the lack of reliable GPU-side timing measurements on Apple Silicon [11, 37].

- ① After the GPU accesses a line that resides in the CPU private caches, subsequent CPU accesses continue to hit in local caches, and no new copy of the line appears in the SLC (Appendix C). This reveals part of the cache coherency: the GPU reads the line

directly from the CPU L2 cache, changing the CPU-resident copy to a shared state [12, 27].

- ② After the GPU accesses data originally residing in the SLC, the CPU's first subsequent access results in an SLC hit, and later accesses yield CPU local cache hits, indicating that GPU access does not invalidate the SLC copy and only changes it to a shared state.
- ③ When the CPU first accesses data previously accessed only by the GPU, the access results in an SLC hit. Subsequent CPU accesses to the same data are then consistently served by the local caches.

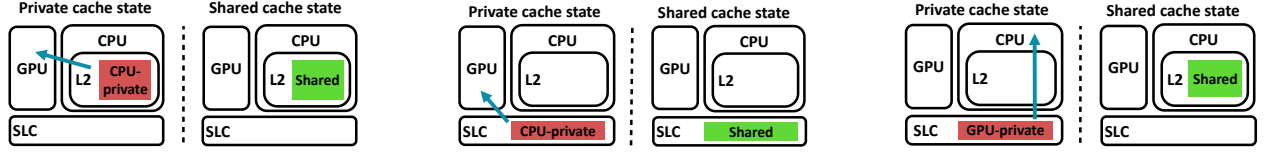
These observations identify two constraints on reliable SLC filling and observation. Scenario ① shows that a GPU access does not evict a line already held in the CPU L2 to SLC, so a GPU read alone cannot always fill an SLC cache line. Scenarios ② and ③ show that the CPU detects SLC residency only on its first access and that all subsequent accesses find the line in the CPU local cache. Therefore, an SLC-residency probe must ensure that the target line is not already cached in the CPU's local cache hierarchy before each measurement.

#### 3.2 Eviction Set Construction

Recovering the set-index function requires eviction sets, that is, groups of addresses that all map to the same cache set. Most prior methods construct eviction sets by iterative pruning. They start from a large candidate pool of memory addresses, repeatedly test whether subsets can evict a reference line, and shrink the pool until only a minimal congruent set remains [9, 10, 17, 28, 35]. The size of this initial pool scales with cache associativity multiplied by  $2^u$ , where  $u$  is the number of set-index bits generated by hashing. In the prior settings for these methods, only a small subset of the index bits is hashed while the remaining bits are drawn directly from low-order physical address bits, so  $u$  is usually fewer than eight, and the candidate pool stays within a few thousand lines.

The Apple M1 SLC uses a substantially different set-indexing mechanism. Prior analysis [32] indicates that none of the set-index bits are drawn directly from the physical address. Section 3.3 shows that all 12 SLC set-index bits are computed by hash functions over multiple address bits, causing the initial candidate pool to expand to tens of thousands of cache lines. Such a large search space makes eviction-set construction highly sensitive to background noise and prevents the standard iterative pruning process from converging reliably.

Instead, we construct eviction sets using *collision-profile clustering*, which replaces single-line eviction testing with a richer per-address signature. The procedure has three steps. We first fill the SLC from the GPU side (details given in Section 4.3) with a filler buffer  $\mathcal{B}_{\text{fill}}$  containing many cache-line-sized memory blocks. We then access a single test cache line from the GPU via another buffer  $\mathcal{B}_{\text{test}}$ , which contends for SLC residency with filled cache lines. After this access, we probe all lines in  $\mathcal{B}_{\text{fill}}$  from the CPU and record which filler lines were evicted. The resulting eviction pattern forms the *collision profile* of that test line. We repeat this measurement for every test line in  $\mathcal{B}_{\text{test}}$ , producing one collision profile per line.



(a) ①: CPU-private (L2-resident) to shared. (b) ②: CPU-private (SLC-resident) to shared. (c) ③: GPU-accessed to shared.

**Figure 3: Three shared-data transition scenarios, showing an example cache line's initial private location (red), the data access that triggers sharing (arrow), and the resulting shared location (green).**

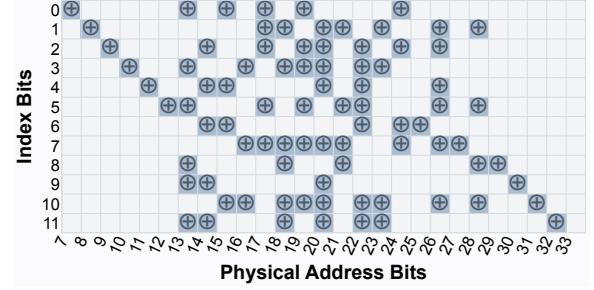
Two test lines that map to the same SLC set will evict the same filler lines and therefore produce highly similar collision profiles. We exploit this property by clustering test lines based on profile similarity. Each cluster corresponds to one SLC set, and the addresses within a cluster form an eviction set. Because each clustering decision aggregates information from many filler-line observations rather than a single hit-or-miss test, this approach is substantially more tolerant to background noise, scheduling jitter, and transient cache-state fluctuations than iterative pruning. The resulting clusters provide reliable inputs for recovering the explicit set-index hash functions.

### 3.3 Recovering SLC Set Indexing Functions

We apply the collision-profile clustering method to a test region  $\mathcal{B}_{\text{test}}$  whose size modestly exceeds the SLC capacity, to ensure that the entire SLC is filled even when the order of filling is non-uniform. We set  $|\mathcal{B}_{\text{test}}|$  to  $1.2 \times$  the SLC size in our experiments. The clustering yields 4,096 clusters, indicating 4,096 SLC sets and therefore a 12-bit set index. We obtain the physical addresses of the cache lines in each cluster through the macOS I/O Kit (IOMemoryDescriptor) and then recover a functionally equivalent set of indexing functions as follows:

- (1) **Identify candidate address bits.** On our 8GB Apple M1 platform, the lower 33 bits compose the effective physical byte address. With 128-byte cache lines, the lowest 7 bits are offset bits, leaving 26 candidate bits contributing to the 12-bit SLC set index.
- (2) **Build candidate function library.** We model each index bit as an XOR over a subset of the 26 candidate bits, yielding a library of  $2^{26}$  candidate bit functions.
- (3) **Filter valid bit functions.** A valid index-bit function must output the same value for all addresses within a cluster, and across 4,096 clusters, it should split evenly (half 0, half 1). This reduces the candidates to 4,095 valid bit functions.
- (4) **Select linearly independent functions.** We select 12 linearly independent valid bit functions that jointly map the 4,096 clusters to distinct 12-bit indices.

Any such set of 12 bit functions is functionally equivalent. The numeric set labels produced by a selected set may differ from the hardware-internal labels, but the recovered mapping deterministically partitions physical addresses into 4,096 distinct SLC sets. Figure 4 shows one set of 12 index bit functions over physical address bits 7–32.



**Figure 4: One set of 12 hashing functions for system-level cache (SLC) set index bits.**

**Replacement policy.** We use a similar process to characterize the SLC replacement policy. We construct a larger eviction set for a selected SLC cache set (twice the cache associativity). We execute GPU kernels that access these lines in controlled orders. CPU probing after each kernel reveals which lines have been evicted. Across configurations, eviction is consistently access-order-dependent: lines accessed earlier are replaced before those accessed more recently, which is consistent with an LRU replacement policy [15, 31].

With the set-indexing function and replacement policy now characterized, we can construct eviction sets that target any SLC set we choose and can predict the eviction order within each set. These two ingredients underpin the CPU-to-GPU side-channel techniques developed in the next section.

## 4 CPU-to-GPU Prime+Probe SLC Side-channel on Apple M1

Building on our reverse-engineering results, we develop an access-driven cache side-channel technique over the Apple M1 System-Level Cache (SLC) with integrated CPU-GPU. The mechanism adapts the classic *Prime+Probe* paradigm to monitor fine-grained SLC set activity induced by GPU workloads.

### 4.1 Overview of the Side-channel

Our threat model includes a sensitive victim workload on the GPU and an adversary on the CPU to carry out SLC priming and probing. The probing phase is always performed on the CPU (CPROBE, Section 4.4) to distinguish SLC hits and misses through accurate access-latency measurements, because Apple GPUs lack reliable fine-grained timing mechanisms [11, 37]. For the priming phase, which populates target SLC sets with attacker-controlled eviction sets, we introduce two alternative designs that differ in efficiency

and in the capabilities required of the attacker. The CPU-based strategy C<sub>PRIME</sub> (Section 4.2) executes entirely on the CPU and does not require the adversary to have GPU access, which makes it easier to deploy under restrictive threat models. When the adversary can also execute on the GPU, the GPU-based strategy G<sub>PRIME</sub> (Section 4.3) exploits GPU parallelism to substantially accelerate priming. With the two different techniques, we build high-bandwidth covert channels and quantify their throughputs (Section 4.5).

## 4.2 Prime with CPU (C<sub>PRIME</sub>)

On Apple M1, the SLC is exclusive with respect to the CPU private caches, so a cache line resides in either the L2 cache or the SLC, but never in both simultaneously. Also, the SLC acts as a backup buffer: when the CPU loads data from memory, it always goes to the L2 cache. Only when an L2 cache set is full, an older cache line is evicted to the SLC due to capacity constraints. To prime the SLC from the CPU, the attacker must load more data than the SLC capacity, using fresher data to evict older data from L2 to fill the SLC. Because the SLC and L2 use different indexing functions, memory blocks chosen to target specific SLC sets are generally mapped to multiple L2 sets, increasing the L2 footprint and consequently slowing the priming process. Considering the exclusiveness, C<sub>PRIME</sub> requires two coordinated data eviction sets: one eventually fills the SLC, and another is the evictor - fresher data that is loaded into L2 while evicting older cache lines to the SLC.

**C<sub>PRIME</sub> setup.** We now describe these two eviction sets in detail. The first set, *SLC\_ES*, aims to fill the entire SLC. With the Apple M1 SLC set associativity of 16 and 4,096 cache sets, *SLC\_ES* contains  $4,096 \times 16 = 65,536$  memory addresses in total (each corresponds to one cache-line size memory block). The second set of memory addresses, *L2\_ES*, tracks all CPU L2 cache sets affected by memory blocks in *SLC\_ES* and aims to fill those sets. Given that the Apple M1 CPU L2 cache set associativity is 12, *L2\_ES* contains  $n_{L_2} \times 12$  memory addresses (cache-line size memory blocks), where  $n_{L_2}$  is the number of affected L2 cache sets.

**Priming procedure.** C<sub>PRIME</sub> proceeds in two steps, as shown in Figure 5. In the first step, the attacker issues one load to each of the 65,536 cache-line addresses in *SLC\_ES*, accessing them in sequence so that every block enters L2. As loading continues, some L2 cache sets reach their 12-way capacity, causing the earliest-loaded blocks at those sets to be evicted from L2 into the SLC. At the end of this step, early-loaded blocks of *SLC\_ES* reside in the SLC, while recent loads reside in the L2 at affected sets. In the second step, the attacker loads *L2\_ES*, which contains 12 addresses for each affected L2 set, which fill the set entirely and push out the surviving *SLC\_ES* data blocks into the SLC. At the end of this step, *SLC\_ES* successfully occupies the SLC in full, completing the priming, while *L2\_ES* occupies the corresponding L2 sets.

**Design optimization.** The only variable of the C<sub>PRIME</sub> process is the number of L2 sets affected by *SLC\_ES* ( $n_{L_2}$ ). We carefully choose *SLC\_ES* so as to reduce  $n_{L_2}$ , lowering the priming time.

A natural approach for constructing an eviction set is to apply the set-indexing functions recovered in Section 3.2 directly to physical addresses. However, macOS does not expose physical addresses to user-space processes. We therefore build both eviction

sets through timing-based primitives and only use the indexing functions to validate them. *L2\_ES* is constructed by the traditional iterative pruning method [9, 10, 17, 28]. This procedure is reliable for L2 because its index uses only a handful of address bits, thereby keeping the candidate pool small. *SLC\_ES* is constructed by the collision-profile clustering method presented in Section 3.2, which identifies set-congruent addresses from the collisions they induce between a test region  $\mathcal{B}_{\text{test}}$  and a filler region  $\mathcal{B}_{\text{fill}}$ .

The L2 cache derives most of its set index bits directly from physical address bits 7–13 and 16–19, while the remaining two index bits are generated through hashing over higher physical address bits[35]. In contrast, all SLC set-index bits are computed through hash functions over multiple physical address bits. Because the SLC hash distributes addresses nearly uniformly, a naively chosen *SLC\_ES* may spread across all 8,192 L2 sets, i.e., yielding a large  $n_{L_2}$  and resulting in *L2\_ES* with  $8,192 \times 12 \approx 98\text{K}$  addresses. Loading such a large eviction set for priming would be prohibitively slow. We reduce this cost by constraining address bits 7–10 to zero when constructing *SLC\_ES*, as they fall within the page offset and can be controlled through virtual addresses alone. Fixing these four bits restricts *SLC\_ES* to  $1/16$  of the L2 cache sets, lowering  $n_{L_2}$  from 8,192 to 512.

Fixing additional address bits would further shrink  $n_{L_2}$ , but at the cost of a larger page footprint for the eviction set. The page size on macOS is 16 KB, holding 128 cache-line blocks (every 128 bytes) with their page offsets specified by the address bits 7–13. Fixing each bit halves the usable memory blocks per page. For example, when bits 7–10 are fixed to zero,  $2^{7-4} = 8$  blocks per page are usable for *SLC\_ES*, and therefore the eviction set spans  $65,536/8 = 8,192$  pages. Fixing an additional bit would increase the page footprint to 16,384 pages. We found that, under this larger page footprint, macOS frequently remaps pages during the attack, invalidating the constructed eviction sets and preventing reliable priming. Such remapping rarely occurs when four bits are fixed. Therefore, we fix this at 4 bits, which reduces the L2 cache set footprint by  $16\times$  while keeping the page count at half the remapping threshold.

## 4.3 Prime with GPU (G<sub>PRIME</sub>)

The attacker can also prime the SLC using a GPU to leverage the massive parallelism, i.e., G<sub>PRIME</sub>. There is another advantage due to asymmetric cache inclusiveness: SLC is exclusive to CPU caches but not exclusive to GPU caches. GPU memory accesses can directly populate the SLC, and therefore, one eviction set *SLC\_ES* is sufficient for priming the SLC. However, a complication arising from cache coherence between the CPU and GPU needs to be addressed.

**Invalidating shared data in CPU L2 from GPU.** Based on findings presented in Section 3.1 (Case 1), if a shared cache-line memory block already has a copy in CPU private caches, GPU read can make a copy in GPU caches through cache coherency without going through the SLC. Therefore, directly reading *SLC\_ES* from the GPU may fail to realize SLC priming when there are cache-line copies in CPU L2. We therefore introduce a GPU-write step to invalidate these stale CPU private-cache copies. However, GPU write-only cannot prime the SLC. Our experiments show that the GPU write operations do not allocate cache lines into the SLC.

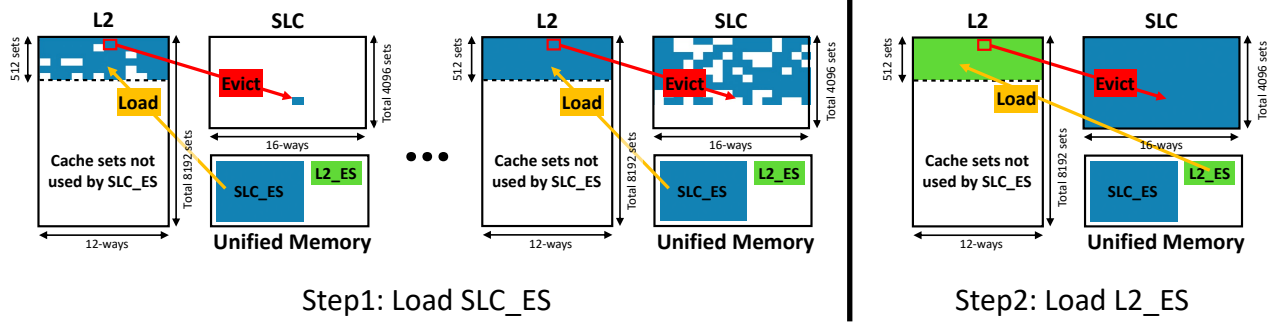


Figure 5: CPROBE procedure: load SLC\_ES first (cache lines only partially in SLC, the rest reside in L2), then load L2\_ES.

This no-write-allocate behavior with SLC is consistent with common GPU cache designs, where write traffic is often bypassed to avoid cache pollution [2, 25]. We further find that even a read-after-write sequence within the same kernel does not reliably bring the data into the SLC. A likely explanation is that the GPU optimizes such producer-consumer data accesses using on-chip tile memory [3], so the subsequent read can be serviced directly from the to-be-written value, rather than by first committing the write to memory and then reloading the data through the SLC path. Based on these findings, we design GPRIME with two GPU kernels: the first kernel writes to all SLC\_ES memory blocks to invalidate any stale CPU private-cache copies or SLC copies, and the second kernel reads the same SLC\_ES addresses to load these memory blocks into the SLC cache lines.

**Facilitating parallel memory accessing.** With the massive parallelism offered by the single-instruction-multiple-thread (SIMT) programming model of GPUs, both GPU-write and GPU-read can be accelerated by processing many data block addresses concurrently. We maintain an auxiliary *index\_sets* structure that stores indices of SLC\_ES addresses for parallel dispatch across GPU threads. This organization is necessary because the method for accessing an eviction set to prime a cache from the CPU does not work for the GPU. The common eviction-set access method is pointer-chasing [10, 17, 28, 31], in which a memory block contains the next address to access, and accesses are sequentially data-dependent, which do not fit thread-level GPU parallelism. To avoid *index\_sets* itself entering the SLC and introducing additional noise, the attacker performs a single pass of CPU reads of *index\_sets* to bring them into the CPU L2 before launching the GPU kernels. During GPRIME kernel execution, the GPU accesses this set via cache coherency, allowing *index\_sets* to remain in CPU L2 throughout the attack without entering the SLC.

Although GPRIME involves both a write kernel and a read kernel, it remains significantly more efficient than CPROBE. In both kernels, one GPU thread is assigned to each address in SLC\_ES, resulting in 65,536 threads per kernel. We set the threadgroup size to 1,024 threads, the maximum supported by the Metal API on Apple M1. Under this configuration, the kernel threads are scheduled in batches of threadgroups and collectively exploit the GPU’s available execution resources at a high degree of parallelism. As a result, GPRIME completes more than an order of magnitude faster than CPROBE, reducing the total priming time from 7 ms to 0.7 ms.

This speedup is attributed to both the massive thread parallelism and the elimination of L2\_ES for GPRIME.

#### 4.4 Probe (CPROBE)

Once the SLC is primed and the victim GPU kernel has finished execution, the attacker must identify the SLC sets accessed by the victim. CPROBE performs this step from the CPU side by re-accessing every memory block in SLC\_ES and timing each access. As our priming process ensures no copies in CPU local caches, each CPROBE access can resolve to either an SLC hit or an SLC miss. An SLC-hit access indicates that the primed line is still resident in the SLC, so the victim did not access any of its own data that maps to this set. An SLC-miss access indicates that the primed line has been evicted, most likely because the victim accessed data mapping to this set, and sometimes because of background system activity.

**Fine-grained occupancy measurement.** Unlike conventional *Prime+Probe* attacks that record only a binary access-or-no-access outcome per cache set, CPROBE counts how many of the 16 primed lines in each SLC set yield high-latency accesses. This per-set integer reveals not only which SLC sets the victim accessed but also how heavily it used each of them, which carries substantially more information than a binary access signal. The final output of CPROBE is therefore an integer trace of length 4,096 (one entry per SLC set), with each element taking a value in  $\{0, 1, \dots, 16\}$  that represents the number of SLC misses observed for the corresponding set.

#### 4.5 Covert Channels

Before applying our side-channel techniques against real GPU workloads in Section 5, we first measure their raw leakage capacity through covert channels. A covert channel is a controlled setting in which the sender and receiver collude to communicate via a microarchitectural medium, in our case, the SLC. This setting removes the confounding effects of a real workload, allowing us to cleanly quantify the throughput, decoding accuracy, and noise tolerance for the two side-channel variants CPROBE+CPROBE and GPRIME+CPROBE.

**Communication protocol.** A GPU kernel acts as the sender, and a CPU process acts as the receiver. In each communication cycle, the receiver first primes a designated SLC set, the sender then modulates the state of that set, and the receiver finally probes the set to recover the transmitted bit. To transmit a ‘1’, the sender fills all 16 cache lines of the target set, resulting in zero SLC hits at

probe time. To transmit a '0', the sender remains idle on that set, so every primed line survives and all 16 produce SLC hits. This full-set modulation maximizes the separation between the two symbols and makes the channel robust to background noise such as OS-induced cache traffic. Under the SLC's LRU replacement policy, the last-primed lines are the most recently used in the set, and any partial eviction from background activity displaces older lines first. If even the last-primed lines are missing at probe time, the set has almost certainly been filled by the sender. The receiver therefore probes only the last five primed addresses, which keeps the probe phase short while preserving decoding reliability.

**Encoding granularity and noise resilience.** In the protocol above, each cache set encodes one bit. The viable encoding granularity, however, depends on the priming method and the noise it introduces. With GPRIME, GPU-based priming populates the SLC in a single uniform pass and produces a clean, low-noise baseline, so one cache set per bit already achieves above 99% bit-level decoding accuracy. GPRIME+CPROBE therefore uses this one-set-per-bit encoding. CPROBE, in contrast, relies on a two-stage priming procedure that involves CPU L2 evictions and is exposed to interference from the OS and CPU private caches. Single-set decoding accuracy falls to 88% under these conditions. To restore reliable transmission, CPROBE+CPROBE assigns four cache sets per bit and has the receiver sum the probe hits across the four sets before decoding. This aggregation improves the decoding accuracy to be above 99%, at the cost of a 4× reduction in bit density per communication cycle. **Synchronization and throughput.** The sender and receiver are synchronized using a millisecond-level timer to align the three phases of priming, transmission, and probing (decoding) within each communication cycle. The two side-channel mechanisms differ in the priming time and transmission density. The priming times reported in Section 4.3 (about 7 ms for CPROBE and 0.7 ms for GPRIME) are run-to-run averages, and individual runs fluctuate around these values. CPROBE likewise takes about 5 ms on average per cycle. To ensure each phase completes before the next begins, even in the worst case, we provision each slot slightly above the average execution time: an 8 ms slot for CPROBE, a 2 ms slot for GPRIME, and a 6 ms slot for CPROBE. In one GPRIME+CPROBE cycle, 4,096 bits are transmitted, while only 1,024 bits are transmitted per CPROBE+CPROBE cycle. Taking into account both the cycle time and the encoding granularity, CPROBE+CPROBE achieves a throughput of 62.5 Kbps, while GPRIME+CPROBE reaches 400 Kbps.

## 5 End-to-end Attacks and Experimental Results

In this section, we demonstrate that our CPU-to-GPU PRIME+PROBE side-channels enable practical privacy attacks that recover secrets of GPU workloads. In Apple Silicon's unified memory architecture, the CPU and GPU share the same physical DRAM and SLC. By monitoring the set-level SLC footprints of the GPU victim workload via our side-channels, the attacker infers the victim's runtime data accesses, thereby leaking the secret.

Our primary attacks build on CPROBE+CPROBE and run entirely from an unprivileged CPU process, without GPU access. We also report alternative attacks based on GPRIME+CPROBE that achieve higher speed and lower noise, but require the attacker to launch GPU kernels via the Apple Metal API.

We first describe the side-channel setup common to both attacks in Section 5.1, including noise-reduction methods for full-SLC monitoring and the three-phase workflow of *profiling*, *trace collection*, and *secret recovery*. We then instantiate this workflow for two privacy attacks on GPU victim workloads: a *GNN edge recovery attack* in Section 5.2 and an *LLM input and output recovery attack* in Section 5.3.

### 5.1 Side-channel Attack Setup

The attacker observes the SLC footprint of a victim GPU kernel, which we call the *target kernel*, by priming and probing the SLC around each kernel invocation. Unless stated otherwise, experiments use CPROBE+CPROBE as the default configuration.

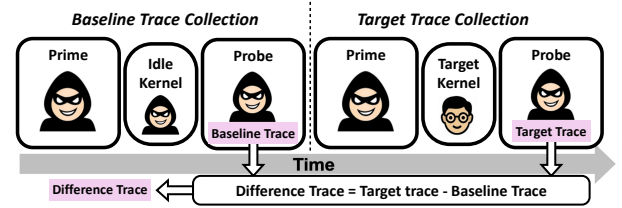
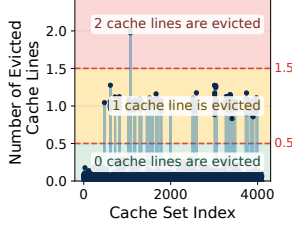


Figure 6: Differential tracing to account for system noise.

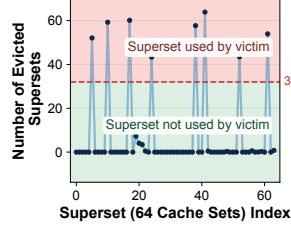
**Differential tracing.** We observe that even in the absence of a victim workload, a small number of cache lines (typically 1 to 4 per set) are evicted by persistent background activities such as display-related work, and these evictions are stable across runs. In addition, when the victim kernel is running, it may leave SLC cache footprints unrelated to the data accesses we aim to observe. To isolate the evictions that the victim's target data accesses actually cause, we adopt *differential tracing*, which collects two traces and takes their difference as the signal of interest. Figure 6 illustrates the procedure. In the *Baseline Trace Collection* phase, the attacker primes the SLC, runs an *idle kernel*, and probes to obtain the baseline trace. The idle kernel invokes the same library function as the victim but excludes the target data accesses (specific setup given per case study in Section 5). The *Target Trace Collection* phase repeats the same prime-run-probe sequence with the actual victim kernel to obtain the target trace. Both traces are CPROBE outputs (integer arrays of length 4,096, described in Section 4.4). Subtracting the baseline from the target trace removes both persistent system noise and victim-side irrelevant activity, leaving only the data access activity of interest. We use differential tracing throughout all subsequent attacks in Section 5.

While differential tracing cancels persistent background noise, we find that additional time-varying noise remains on top of this baseline and requires further mitigation. We counter this noise with two methods that restore the decoding reliability: multi-run averaging and cache-set aggregation.

**Method 1: Multi-run averaging.** The first method reduces measurement variance by averaging the side-channel traces. Each run records a per-set SLC-miss count, an integer between 0 and 16. Averaging these counts across runs preserves the victim's footprint while smoothing out run-to-run fluctuations. Figure 7 shows an averaged CPROBE+CPROBE trace collected while the victim runs a Graph Neural Network (GNN) kernel (Section 5.2). The averaged



**Figure 7: Averaged side-channel trace (1000 runs) for 4096 cache sets.**



**Figure 8: Aggregated single-trial side-channel trace for 64 supersets.**

values observed in this trace cluster cleanly around 0, 1, and 2 occupied lines per set, and we separate these clusters using midpoint thresholds of 0.5 and 1.5. To quantify accuracy, we define *undercounting* and *overcounting* as the fractions of cache sets whose inferred occupancy falls below or above the ground truth. Table 1 (Columns 2-4) reports these rates alongside the number of averaging runs. CPRIME+CPROBE needs 4,000 runs to match the error rate that GPRIME+CPROBE achieves in 1,000 runs, because the two-stage CPU priming phase is inherently noisier than direct GPU priming.

**Table 1: Accuracies with the two mitigations**

| Priming Method | Method1    |           |       | Method2 |       |            |
|----------------|------------|-----------|-------|---------|-------|------------|
|                | Undercount | Overcount | #Runs | FNR     | FPR   | #Supersets |
| CPRIME         | 0.04%      | 0.53%     | 4000  | 0.17%   | 2.63% | 64         |
| GPRIME         | 0.02%      | 0.37%     | 1000  | 0.08%   | 1.03% | 64         |

**Method 2: Cache-set aggregation (superset).** The second method trades per-set resolution for a cleaner single-trial signal. Instead of treating each of the 4,096 SLC sets independently, we partition them into equal-size groups, called *superset*. For each *superset* we report the aggregate number of victim-occupied lines. This aggregation is effective whenever each candidate victim data element is a multiple of (a power-of-two) cache lines. Figure 8 shows a single-trial CPRIME+CPROBE trace with 64 supersets of 64 sets each. An accessed superset reports a value close to 64, one per set, while an untouched superset stays near 0, so a midpoint threshold of 32 cleanly separates the two outcomes. Because aggregation only reveals whether a superset is accessed, we generate a binary access trace. Table 1 (Columns 5-7) reports the corresponding False-Negative Rate (FNR) and False-Positive Rate (FPR) over 64 supersets, with both priming strategies keeping single-trial errors below 3%.

**Three-phase attack methodology.** Both the GNN and LLM attacks share a common three-phase workflow with their respective noise-reduction method. We model the victim’s GPU workload as accessing items drawn from a finite *Target Item List*  $\mathcal{L} = \{i_1, i_2, \dots, i_N\}$ , where each item corresponds to a data structure in memory such as a feature vector, a lookup-table entry, or a matrix row. Figure 9 illustrates the general three-phase workflow along with its two instantiations: the GNN edge recovery attack (left panel) and the LLM input and output recovery attack (right panel). The three phases are as follows.

- (1) **Profiling.** The attacker profiles each item  $i_i$  individually to build an *Access Mapping Matrix*  $M^{N \times S}$  for the entire list  $\mathcal{L}$ , where  $S$  is the number of SLC cache sets (Method 1) or supersets

(Method 2). Under Method 1,  $M_{i,j} \in \{0, 1, \dots, 16\}$  is the cache line occupancy of item  $i_i$  in set  $j$ . Under Method 2,  $M_{i,j} \in \{0, 1\}$  indicates superset  $j$  no-access versus access by item  $i_i$ .

- (2) **Trace collection.** The attacker obtains an SLC side-channel trace for each victim kernel invocation, denoted as an *Observed Access Vector*  $\mathbf{v} \in \mathbb{Z}^S$ . Under Method 1,  $v_j \in \{0, 1, \dots, 16\}$  is the number of victim-occupied lines in cache set  $j$ . Under Method 2,  $v_j \in \{0, 1\}$  is the observed superset  $j$  access flag.
- (3) **Secret recovery.** The attacker compares  $\mathbf{v}$  against the rows of  $M$  to infer which items in  $\mathcal{L}$  the victim accessed during that invocation. The specific comparison algorithm depends on the attack and is detailed in Sections 5.2 and 5.3.

## 5.2 GNN Edge Recovery Attack

Graph Neural Networks (GNNs) are commonly used in recommendation systems and social networks. For node inference, a target node’s representation is computed by aggregating the features of its neighbor nodes. In applications such as social networks and biomedical analysis, the graph’s edge structure encodes proprietary or privacy-critical relationships. Because the neighbors accessed during inference directly reflect these edges, the memory access pattern becomes a high-value target for privacy attacks.

**Threat model.** A GNN is deployed on an Apple M-series SoC using PyTorch with Metal GPU acceleration. The attacker is a co-resident, unprivileged CPU process that can issue arbitrary node queries to the model for inference, a common chosen-query setting [18, 38]. The attacker’s goal is to recover the full set of graph edges from side-channel observations collected during these queries. We evaluate both CPRIME+CPROBE and GPRIME+CPROBE attacks.

**Deployment characteristics.** When the model is deployed, PyTorch allocates the feature vectors of all graph nodes in the unified memory via Apple’s Metal API. During inference on a target node, the model invokes `Torch.index_select()` to gather its neighbors’ feature vectors into a temporary tensor for aggregation. This gather operation is the source of side-channel leakage: each neighbor’s feature vector occupies a distinct region of memory, and the GPU’s read operations leave clear footprints on the SLC that a CPU-side attacker can monitor. Accordingly, the idle kernel used for differential tracing (Section 5.1) invokes the `Torch.index_select()` call with an empty index set.

**Attack workflow.** We instantiate the general three-phase attack workflow for GNN edge recovery, treating each graph node’s feature vector as an item  $i_i$  in the Target Item List  $\mathcal{L}$ . The left panel of Figure 9 illustrates the workflow.

- (1) **Profiling.** The attacker constructs a binary Access Mapping Matrix  $M_G \in \{0, 1\}^{N \times 4096}$ , where each row corresponds to one graph node feature vector, recording which SLC cache sets the feature vector maps to. Because macOS does not expose the physical addresses of GPU-resident tensors to user-space processes, the attacker cannot directly apply the SLC indexing function from Section 3.2 and instead profiles  $M_G$  empirically with the full SLC priming and probing.
- (2) **Trace collection.** During a target-node query, the GNN kernel accesses all of that node’s neighbors’ feature vectors in a single batch, so the resulting Observed Access Vector  $\mathbf{v} \in \mathbb{Z}^{4096}$  is the superposition of all neighbors’ SLC footprints. Each entry

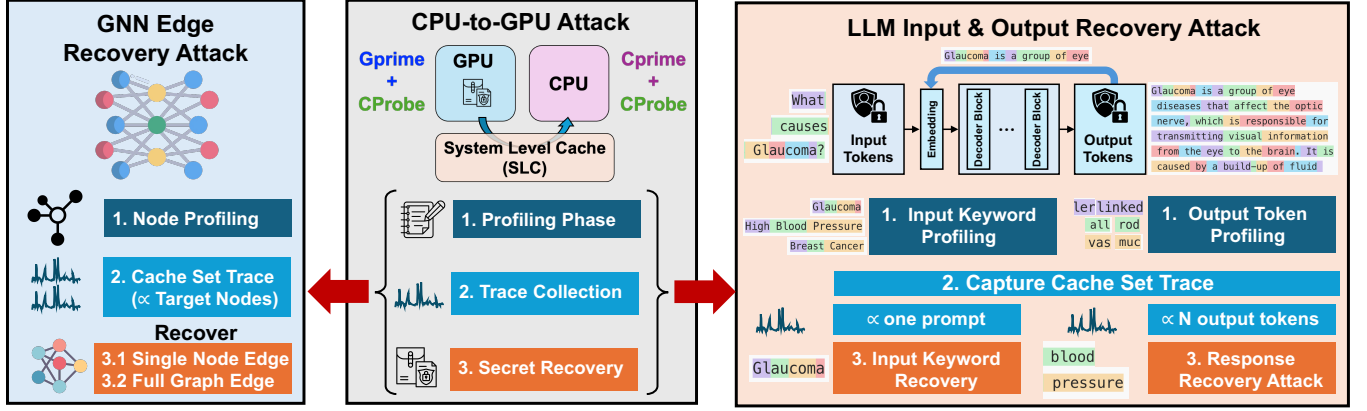


Figure 9: Proposed CPU-to-GPU cache side-channel attack workflow using CPRIME+CPROBE and GPRIME+CPROBE, with its application to GNN edge recovery and LLM input/output recovery attacks.

$v_j \in \{0, 1, \dots, 16\}$  counts the cache lines in set  $j$  the victim accesses. We collect  $\mathbf{v}$  with Method 1 (multi-run averaging) to reduce the system noise.

- (3) **Secret recovery.** The attacker compares  $\mathbf{v}$  against the rows of  $\mathcal{M}_G$  to infer which nodes the victim accessed during the target-node query, and each accessed node yields one edge of the target node. We measure the recovery quality using *precision* (fraction of reported edges that are true) and *recall* (fraction of true edges that are recovered). The recovery proceeds in two stages as described next.

**Single-node edge recovery.** Given the access vector  $\mathbf{v}$  for a target node  $n$ , the attacker recovers its neighbors by solving a *set cover* problem over the rows of  $\mathcal{M}_G$ : identifying the minimal collection of node profiles whose combined cache-set footprint matches  $\mathbf{v}$ . The attacker first filters  $\mathcal{M}_G$  to a candidate set  $\hat{C}$  of profiles that are element-wise dominated by  $\mathbf{v}$ , namely those satisfying  $\mathcal{M}_G[i, j] \leq \mathbf{v}[j]$  at every cache-set index  $j$ . The attacker then prunes  $\hat{C}$  down to the intersection of all size- $\hat{m}$  subsets whose combined footprint matches  $\mathbf{v}$ , where  $\hat{m}$  is the neighbor count estimated from the total occupancy in  $\mathbf{v}$ . Pruning is invoked only when  $\hat{m}$  falls below a dataset-specific trustworthiness threshold  $T$  (Table 2), and the attacker otherwise returns  $\hat{C}$  unchanged. The full algorithm, including the derivation of  $T$  and the additional fallback paths, is given in Appendix D.

**Full-graph edge recovery.** After recovering a candidate neighbor set for each node, the attacker reconstructs the entire edge set by aggregating these per-node candidates. Because the graph is undirected, a true edge  $(u, v)$  must appear in the candidate sets of both  $u$  and  $v$ , so the attacker keeps only edges confirmed from both endpoints. False positives rarely confirm bidirectionally, so this check improves precision while preserving high recall.

**Evaluation.** We evaluate the GNN edge recovery attack on a Graph Convolutional Network (GCN) [4] over five benchmark datasets: Cora, Citeseer, Pubmed, Computers, and Photo [22, 34]. Table 2 summarizes the dataset characteristics and reports the attack accuracy under both CPRIME+CPROBE and GPRIME+CPROBE in terms of precision and recall.

With CPRIME+CPROBE, the CPU-only attacker achieves strong results. Single-node recall is consistently high, typically above 94% and often exceeding 98%, which reflects the low undercounting rate of the side-channel. Single-node precision, however, varies with graph density. Sparse graphs (Cora, Citeseer, Pubmed) keep precision in the 83-92% range because nodes have few neighbors and per-neighbor cache footprints rarely overlap. Dense graphs (Computers, Photo) drop in precision because the larger neighborhoods produce extensive cache-set overlap and inflate  $C$  with spurious candidates. The full-graph recovery stage substantially closes this gap, raising CPRIME+CPROBE precision above 86% on all datasets while keeping recall above 90%. GPRIME+CPROBE attacks achieve higher precision and recall because GPU-based priming yields a cleaner SLC state. Full-graph precision then exceeds 99% on Cora, Citeseer, and Pubmed.

### 5.3 LLM Privacy Attacks

Large Language Models (LLMs), particularly decoder-only architectures such as GPT and Llama, are now the dominant paradigm for language-based inference. A decoder-only model consists of an embedding layer, a stack of transformer decoder blocks, and a projection layer. The embedding layer looks up a table in memory for the given input (prompt token or the previously output response token), whose rows store the fixed-dimensional embedding of each token in the vocabulary. Because these reads are input-dependent and span both the *prefill* and *decoding* phases, the embedding layer forms a natural side-channel attack surface revealing private queries, proprietary data, or confidential inferences. The embedding layer aligns with the *Target Item List* abstraction in Section 5.1: each token embedding corresponds to an item  $i_i$ , and the attacker’s goal is to infer which embeddings the victim accesses by observing SLC eviction patterns.

**Threat model.** The victim deploys a decoder-only LLM on the GPU of an Apple M-series SoC using PyTorch with Metal GPU acceleration. The attacker is a co-resident, unprivileged background process on the same system. Although the LLM model is provided as a shared library on the platform, the victim runtime information, including the victim’s input prompts and the generated responses, is kept private with process isolation. The attacker only relies on

**Table 2: Datasets and GNN edge recovery attack results, reported as precision/recall (P/R). P is the percentage of reported edges that are true, and R is the percentage of true edges recovered. Columns are shaded by attack: CPRIME and GPRIME.**

| Dataset   | #Nodes (N) | #Edges  | #Feat. | Avg. # of cache lines per node ( $\bar{f}$ ) | Threshold $T$ |        | Single-Node Recovery |              | Full-Graph Recovery |              |
|-----------|------------|---------|--------|----------------------------------------------|---------------|--------|----------------------|--------------|---------------------|--------------|
|           |            |         |        |                                              | CPRIME        | GPRIME | CPRIME (P/R)         | GPRIME (P/R) | CPRIME (P/R)        | GPRIME (P/R) |
| Cora      | 2,708      | 10,556  | 1,433  | 46.78                                        | 60            | 95     | 84.3 / 97.2          | 90.6 / 98.8  | 98.2 / 94.2         | 99.9 / 97.3  |
| Citeseer  | 3,327      | 9,104   | 3,703  | 117.72                                       | 74            | 105    | 91.6 / 94.2          | 95.2 / 97.3  | 97.8 / 90.5         | 99.2 / 94.1  |
| Pubmed    | 19,717     | 88,648  | 500    | 17.63                                        | 28            | 40     | 82.9 / 99.1          | 83.2 / 99.6  | 99.8 / 97.9         | 99.9 / 98.8  |
| Computers | 13,752     | 491,722 | 767    | 25.97                                        | 53            | 93     | 30.8 / 98.5          | 33.2 / 99.2  | 86.6 / 96.8         | 87.2 / 98.6  |
| Photo     | 7,650      | 238,162 | 745    | 25.29                                        | 36            | 58     | 46.6 / 98.5          | 48.3 / 99.3  | 93.8 / 97.2         | 94.9 / 98.6  |

the SLC side-channel to observe the embedding-layer footprints left by the victim’s execution.

**Deployment characteristics.** PyTorch loads the token embedding table into unified memory via Apple’s Metal API, and the embedding layer is invoked on every forward pass. The GPU’s read operations leave clear footprints on the SLC that a CPU-side attacker can monitor, making the embedding-layer lookup the *target kernel* of our attack. Accordingly, the *idle kernel* used for differential tracing (Section 5.1) invokes the same embedding layer with an empty input token set, so it exercises the same code path as the target kernel without performing any token-driven embedding lookups.

We evaluate two applications, in which the victim queries the LLM about one of the 1,000 most common diseases (drawn from the MEDQUAD [1] prompt dataset) or one of the top 100 publicly traded stock tickers (drawn from the FINANCIALQA [21] prompt dataset). In each case, the attacker maintains a matching domain-specific dictionary of disease names or stock tickers.

**Attack overview.** We instantiate the three-phase attack workflow of Section 5.1 in two complementary attacks. The first, *input keyword recovery*, targets the *prefill* phase, in which the model processes the prompt and looks up embeddings for all input tokens in a single batch, aiming to identify sensitive keywords or topics in the user’s prompt. The second, *response recovery*, targets the *decoding* phase, in which the model generates the output autoregressively and accesses the embedding layer once per step to look up the previously generated token, aiming to reconstruct the model’s output token-by-token. The right panel of Figure 9 shows both attacks.

**Superset setting.** Unlike the GNN attack, which monitors all 4,096 individual SLC cache sets, the LLM attack groups cache sets into *supersets* and monitors at a coarser granularity, following Method 2 of Section 5.1 to reduce side-channel noise. This aggregation is applicable to LLM inference because embedding tables have a highly regular memory layout. Each token embedding is a fixed-size vector whose dimension is typically a power of two (e.g., 1024 or 2048), and embedding layers are commonly page-aligned so that the first vector starts at page offset zero [20, 36]. Each embedding, therefore, occupies a contiguous, aligned region of physical memory.

The alignment of embeddings interacts predictably with the SLC set-indexing functions. Figure 4 shows that on Apple M1, for the lower 6 SLC set index bits, each is directly obtained from one address bit (within 7–12). Now consider an embedding of size  $2^m$  bytes with  $7 < m < 12$ . For its memory block addresses, the address bits below  $m$  vary across cache-line blocks, whereas bits  $m$  and above are fixed. Applying the indexing functions, only the lower  $m - 7$  SLC index bits sweep through all values for the embedding,

while the higher index bits stay constant. Each embedding thus maps to a contiguous, power-of-two-aligned block of  $2^{m-7}$  cache sets that does not overlap with any other embedding’s block. These blocks are supersets that partition the 4,096 SLC sets into a lower dimension, with the side-channel length reduced from 4,096 to 128 or 256, depending on the embedding size.

We profile two LLMs, TinyLlama [36] and GPT-2 Medium [20]. Table 3 summarizes their structural characteristics, the token embedding memory size, cache-line count per token, and the resulting number of supersets.

**Table 3: Memory and SLC Usage of LLMs Token Embedding.**

| Model            | #Param | Vocab Size | Memory /Token | #Lines /Token | # Supersets |
|------------------|--------|------------|---------------|---------------|-------------|
| TinyLlama[36]    | 1.1B   | 32,000     | 4KB           | 32            | 128         |
| GPT-2 Medium[20] | 0.35B  | 50,257     | 2KB           | 16            | 256         |

**5.3.1 Input Keyword Recovery Attack.** This attack targets the *prefill* phase of LLM inference to identify the sensitive keyword in the user’s prompt (e.g., a disease name or stock ticker) by monitoring the SLC evictions caused by embedding-layer lookups.

- (1) **Profiling.** The attacker profiles the SLC superset access pattern for each keyword in the domain-specific dictionary, treating each keyword as a sequence of one or more tokens. Profiling all  $n$  keywords produces a binary matrix  $M_K \in \{0, 1\}^{n \times S}$ , where  $S$  is the number of supersets and each row records which supersets a given keyword maps to. When the tokens of a keyword map to disjoint supersets, the row’s active-superset count equals the token count. Most keywords in our dictionaries span 3–8 tokens.
- (2) **Trace collection.** The attacker monitors the SLC supersets during the victim’s prefill phase and records a binary vector  $\mathbf{v}_{\text{input}} \in \{0, 1\}^S$  in which each entry indicates whether the corresponding superset was accessed. Because the prompt contains additional tokens beyond the target keyword (function words, punctuation, and so on),  $\mathbf{v}_{\text{input}}$  is generally denser than the keyword’s own superset profile.
- (3) **Secret recovery.** The attacker compares  $\mathbf{v}_{\text{input}}$  against each row of  $M_K$  and flags every keyword whose superset profile is fully contained in  $\mathbf{v}_{\text{input}}$  as a candidate. Ideally, this yields a unique candidate, since each prompt contains exactly one keyword from the dictionary. However, supersets activated by non-keyword tokens may also satisfy containment for unrelated keywords, introducing false positives. When multiple candidates remain, the attacker selects one randomly.

**Table 4: Accuracy of LLM Input Keyword Recovery**

| Dataset     | TinyLlama |        | GPT-2 Medium |        |
|-------------|-----------|--------|--------------|--------|
|             | CPRIME    | GPRIME | CPRIME       | GPRIME |
| MedQuad     | 76.3%     | 91.5%  | 78.2%        | 93.4%  |
| FinancialQA | 80.5%     | 94.2%  | 81.2%        | 94.8%  |

Table 4 reports keyword recovery accuracy, 76–81% across the two datasets and two models for CPRIME+CPROBE, rising to 91–95% for GPRIME+CPROBE, reflecting the lower noise floor of GPRIME.

**5.3.2 Response Recovery Attack.** Unlike the input keyword recovery attack, which targets the *prefill* phase, response recovery operates token-by-token during the autoregressive *decoding*, which yields cleaner per-token side-channel observations and requires a simpler recovery algorithm. The attack uses the same three-phase workflow.

- (1) **Profiling.** The attacker profiles individual token embeddings rather than multi-token keywords. As the full vocabularies are prohibitively large (see Table 3), we restrict profiling to the 3,000 most frequent tokens, which cover over 95% of tokens in our evaluation prompts. Profiling produces a binary matrix  $\mathcal{M}_O \in \{0, 1\}^{T \times S}$ , where each row records the superset mapping of one of the  $T$  profiled tokens across  $S$  supersets.
- (2) **Trace collection.** At each decoding step, the embedding layer looks up exactly one token embedding and activates a single SLC superset, which the attacker reads as the superset with the maximum observed access value. For a response of  $r$  output tokens, the attacker therefore collects  $r - 1$  traces. This direct superset identification achieves high accuracy on both models under both priming strategies: 99.2% (CPRIME) and 99.7% (GPRIME) on TinyLlama, and 98.8% (CPRIME) and 99.5% (GPRIME) on GPT-2 Medium. GPT-2 Medium scores slightly lower than TinyLlama because its larger embedding table spans more supersets, raising the chance of superset-level ambiguity.
- (3) **Secret recovery.** Although the attacker identifies the accessed superset at each decoding step with high accuracy,  $\mathcal{M}_O$  has only 128 or 256 supersets but covers about 3,000 profiled tokens, so each superset maps to many tokens. Each observed superset, therefore, yields a candidate set  $C_t$  rather than a unique token. To resolve this ambiguity, the attacker uses a local copy of the same LLM as a language model. Given the prefix of already-recovered tokens, the attacker runs the LLM to obtain logits over the full vocabulary, restricts the softmax to the candidates in  $C_t$ , and selects the most probable token.

**Bootstrapping the initial output tokens.** Initially, the prefix is either empty or too short to disambiguate early candidates. To handle this, the attacker jointly recovers the first three tokens by enumerating all tuples  $(x_1, x_2, x_3)$  with  $x_i \in C_i$  and scoring each tuple as:

$$\text{Score}(x_1, x_2, x_3) = P_{\text{LLM}}(x_1 \mid \emptyset) \cdot P_{\text{LLM}}(x_2 \mid x_1) \cdot P_{\text{LLM}}(x_3 \mid x_1, x_2), \quad (1)$$

where each probability term comes from running the LLM on the corresponding prefix. The tuple with the highest score becomes the starting prefix.

**Table 5: Accuracy of LLM Output Response Recovery**

| Dataset     | TinyLlama |        | GPT-2 Medium |        |
|-------------|-----------|--------|--------------|--------|
|             | CPRIME    | GPRIME | CPRIME       | GPRIME |
| MedQuad     | 70.5%     | 75.7%  | 79.2%        | 83.6%  |
| FinancialQA | 73.4%     | 78.7%  | 85.3%        | 88.9%  |

The choice of three jointly recovered tokens trades accuracy against search cost. Larger windows provide the language model with more context, but they also exponentially increase the search space with respect to the number of positions. In our experiments, going beyond three tokens yields only marginal accuracy improvements while increasing the search cost by a factor of tens, so three is the smallest window that captures the accuracy benefit.

**Iterative decoding for the remaining tokens.** Once the first three tokens are fixed, the attacker recovers each subsequent token  $t \geq 4$  greedily by selecting the candidate in  $C_t$  with the highest probability under the recovered prefix:

$$\hat{x}_t = \arg \max_{x \in C_t} P_{\text{LLM}}(x \mid \hat{x}_1, \dots, \hat{x}_{t-1}). \quad (2)$$

Table 5 reports the response recovery accuracy, defined as the fraction of output tokens correctly recovered across the response. CPRIME+CPROBE attacks recover 70–85% of output tokens correctly across the two models and two datasets. GPRIME+CPROBE attacks improve the accuracy by 4–5%. Because each decoded token conditions all subsequent predictions, early errors propagate through the autoregressive chain, leading to cascading mispredictions. This error amplification explains why the CPRIME-versus-GPRIME gap is larger here than at the superset level: although CPRIME’s superset accuracy is less than 1% below GPRIME’s, the small per-step gap compounds across sequential decoding. GPT-2 Medium scores higher than TinyLlama on response recovery despite a slightly lower superset accuracy, because its larger embedding table distributes tokens across more supersets, yielding fewer candidates per superset and making disambiguation more reliable.

## 6 Conclusion and Future Work

This paper presents SLAC, the first fine-grained Prime+Probe CPU-to-GPU cache side-channel attack on Apple Silicon. By reverse-engineering the Apple M1 SLC, including its 12-bit set-indexing hash functions, we constructed two side-channel techniques. CPRIME+CPROBE lets an unprivileged CPU process observe GPU victim activity, and GPRIME+CPROBE uses GPU-side priming for a higher attack speed. The corresponding covert channels achieved throughputs of 62.5 Kbps and 400 Kbps, respectively. We demonstrated two end-to-end privacy attacks based on the new side-channels: a GNN edge recovery attack with over 90% accuracy across five benchmark datasets, and an LLM privacy attack on TinyLlama and GPT-2 Medium that recovered input keywords with up to 94.8% accuracy and output tokens with up to 88.9% accuracy.

The broader takeaway is that the shared SLC in Apple’s unified memory architecture is a potent cross-domain attack surface, enabling fine-grained leakage of sensitive GPU workloads to an unprivileged CPU process. As heterogeneous CPU-GPU SoCs increasingly host privacy-critical ML workloads, these findings motivate

careful redesign of cache hierarchies and coherence protocols to mitigate access-driven side-channels across compute domains.

**Future work.** The first direction is broadening this attack family across the Apple M-series. M1 Pro/Max and M2 share the same overall architecture as M1, including separate CPU and GPU clusters, a shared SLC, and the asymmetric SLC residency that our side-channel techniques exploit, so we expect both C<sub>PRIME</sub>+C<sub>PROBE</sub> and G<sub>PRIME</sub>+C<sub>PROBE</sub> to apply with only platform-specific calibration, such as re-recovering the SLC set-indexing functions. The Apple M3 SoC, however, presents additional challenges: it introduces Dynamic Caching for GPU memory, which could affect interactions between local caches and the SLC.

The second direction is discovering cross-domain side-channels on other heterogeneous platforms. Intel iGPUs [6] and AMD APUs [30] use non-inclusive GPU cache policies, so the residency asymmetry we exploit on Apple Silicon manifests differently on those designs and opens a complementary set of shared channels. Promising candidates include memory-controller occupancy, interconnect contention, and Prime+Probe variants tailored to each vendor's inclusiveness rules, all of which would extend cross-domain side-channel analysis to the broader heterogeneous SoC landscape.

## Acknowledgments

This work was supported in part by National Science Foundation under grant CNS-1916762 and IUCRC Center for Hardware and Embedded Systems Security and Trust (CHEST).

## References

- [1] Afroz. 2023. MEDQUAD: Medical Question Answer for AI Research. <https://www.kaggle.com/datasets/pythonaifroz/medquad-medical-question-answer-for-ai-research>. <https://www.kaggle.com/datasets/pythonaifroz/medquad-medical-question-answer-for-ai-research>
- [2] Neha Agarwal, David Nellans, Eiman Ebrahimi, Thomas F Wenisch, John Danskin, and Stephen W Keckler. 2016. Selective GPU caches to eliminate CPU-GPU HW cache coherence. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 494–506. doi:10.1109/HPCA.2016.7446089
- [3] Apple Inc. [n. d.]. Tailor Your Apps for Apple GPUs and Tile-Based Deferred Rendering. <https://developer.apple.com/documentation/metal/tailor-your-apps-for-apple-gpus-and-tile-based-deferred-rendering>. Accessed: 2026-04-01.
- [4] Rianne van den Berg, Thomas N Kipf, and Max Welling. 2017. Graph convolutional matrix completion. *arXiv preprint arXiv:1706.02263* (2017). doi:10.48550/arXiv.1706.02263
- [5] Patrick Cronin, Xing Gao, Haining Wang, and Chase Cotton. 2021. An exploration of ARM system-level cache and GPU side channels. In *Proceedings of the 37th Annual Computer Security Applications Conference*. 784–795. doi:10.1145/3485832.3485902
- [6] Sankha Baran Dutta, Hoda Naghibijouybari, Nael Abu-Ghazaleh, Andres Marquez, and Kevin Barker. 2021. Leaky buddies: Cross-component covert channels on integrated cpu-gpu systems. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 972–984. doi:10.1109/ISCA52012.2021.00080
- [7] Daniel Genkin, Lev Pachmanov, Eran Tromer, and Yuval Yarom. 2018. Drive-by key-extraction cache attacks from portable code. In *International Conference on Applied Cryptography and Network Security*. Springer, 83–102. doi:10.1007/978-3-319-93387-0\_5
- [8] Lukas Giner, Stefan Steinegger, Antoon Purnal, Maria Eichlseder, Thomas Unterluggauer, Stefan Mangard, and Daniel Gruss. 2023. Scatter and split securely: Defeating cache contention and occupancy attacks. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2273–2287. doi:10.1109/SP46215.2023.10179440
- [9] Daniel Gruss, Clémentine Maurice, and Stefan Mangard. 2016. Rowhammer.js: A remote software-induced fault attack in javascript. In *International conference on detection of intrusions and malware, and vulnerability assessment*. Springer, 300–321. doi:10.1007/978-3-319-40667-1\_15
- [10] Ralf Hund, Carsten Willems, and Thorsten Holz. 2013. Practical timing side channel attacks against kernel space ASLR. In *2013 IEEE Symposium on Security and Privacy*. IEEE, 191–205. doi:10.1109/SP.2013.23
- [11] Zhen Hang Jiang, Yungsi Fei, and David Kaeli. 2016. A complete key recovery timing attack on a GPU. In *2016 IEEE International Symposium on high performance computer architecture (HPCA)*. IEEE, 394–405. doi:10.1109/HPCA.2016.7446081
- [12] Harshavardhan Kaushikkar, Christopher D. Shuler, Srinivasa Rangan Sridharan, Yu Zhang, Kaushik Kannan, and Deniz Balkan. 2021. Inter Cluster Snoop Latency Reduction. U.S. Patent No. 11,016,913 B1. <https://patents.google.com/patent/US11016913B1/en> Filed Mar. 30, 2020; issued May 25, 2021. Assignee: Apple Inc..
- [13] Taehun Kim, Jaehun Kim, and Youngjoo Shin. 2021. Constructing Covert Channel on Intel CPU-iGPU platform. In *2021 International Conference on Information Networking (ICOIN)*. IEEE, 39–42. doi:10.1109/ICOIN50884.2021.9333985
- [14] Michael Kurth, Ben Gras, Dennis Andriesse, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. 2020. NetCAT: Practical cache attacks from the network. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 20–38. doi:10.1109/SP40000.2020.00082
- [15] Donghee Lee, Jongmoo Choi, Jong-Hun Kim, Sam H Noh, Sang Lyul Min, Yookun Cho, and Chong Sang Kim. 1999. On the existence of a spectrum of policies that subsumes the least recently used (LRU) and least frequently used (LFU) policies. In *Proceedings of the 1999 ACM SIGMETRICS international conference on Measurement and modeling of computer systems*. 134–143. doi:10.1145/301453.301487
- [16] Moritz Lipp, Daniel Gruss, Raphael Spreitzer, Clémentine Maurice, and Stefan Mangard. 2016. {ARMeddion}: Cache attacks on mobile devices. In *25th USENIX Security Symposium (USENIX Security 16)*. 549–564. <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/lipp>
- [17] Bradley Morgan, Gal Horowitz, Sioli O'Connell, Stephan van Schaik, Chitchanok Chuengsatiansup, Daniel Genkin, Olaf Maennel, Paul Montague, Eyal Ronen, and Yuval Yarom. 2025. Slice+ Slice Baby: Generating Last-Level Cache Eviction Sets in the Blink of an Eye. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3479–3496. doi:10.1109/SP61157.2025.00264
- [18] Marcin Podhajski, Jan Dubiński, Franziska Boenisch, Adam Dziedzic, Agnieszka Pręgoska, and Tomasz Paweł Michalak. 2026. On Stealing Graph Neural Network Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 24846–24854. doi:10.1609/aaai.v40i29.39671
- [19] Antoon Purnal, Furkan Turan, and Ingrid Verbauwhede. 2021. Prime+ Scope: Overcoming the observer effect for high-precision cache contention attacks. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2906–2920. doi:10.1145/3460120.3484816
- [20] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9. <https://openai.com/blog/better-language-models/>
- [21] Yousef Saeedian. 2023. Financial Q&A 10K. <https://www.kaggle.com/datasets/yousefsaeedian/financial-q-and-a-10k>. <https://www.kaggle.com/datasets/yousefsaeedian/financial-q-and-a-10k>
- [22] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018). doi:10.48550/arXiv.1811.05868
- [23] Anatoly Shusterman, Ayush Agarwal, Sioli O'Connell, Daniel Genkin, Yossi Oren, and Yuval Yarom. 2021. {Prime+ Probe} 1, {JavaScript} 0: Overcoming Browser-based {Side-Channel} Defenses. In *30th USENIX Security Symposium (USENIX Security 21)*. 2863–2880. <https://www.usenix.org/conference/usenixsecurity21/presentation/shusterman>
- [24] Anatoly Shusterman, Zohar Avraham, Eliezer Croitoru, Yarden Haskal, Lachlan Kang, Dvir Levi, Yosef Meltser, Prateek Mittal, Yossi Oren, and Yuval Yarom. 2020. Website fingerprinting through the cache occupancy channel and its real world practicality. *IEEE Transactions on Dependable and Secure Computing* 18, 5 (2020), 2042–2060. doi:10.1109/TDSC.2020.2988369
- [25] Inderpreet Singh, Arrvinth Shriraman, Wilson WL Fung, Mike O'Connor, and Tor M Aamodt. 2013. Cache coherence for GPU architectures. In *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 578–590. doi:10.1109/HPCA.2013.6522351
- [26] Yan Solihin. 2015. *Fundamentals of Parallel Multicore Architecture* (1st ed.). Chapman & Hall/CRC. doi:10.1201/b20200 Chapter 5.
- [27] James Vash, Gaurav Garg, Brian P. Lilly, Ramesh B. Gunna, Steven R. Hutsell, Lital Levy-Rubin, Per H. Hammarlund, and Harshavardhan Kaushikkar. 2024. Scalable Cache Coherency Protocol. U.S. Patent No. 11,947,457 B2. <https://patents.google.com/patent/US11947457B2/en> Filed Nov. 22, 2022; issued Apr. 2, 2024. Assignee: Apple Inc. Continuation of U.S. Patent No. 11,544,193.
- [28] Pepe Vila, Boris Köpf, and José F Morales. 2019. Theory and practice of finding eviction sets. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 39–54. doi:10.1109/SP.2019.00042
- [29] Han Wang, Syed Mahbub Hafiz, Kartik Patwari, Chen-Nee Chuah, Zubair Shafiq, and Houman Homayoun. 2022. Stealthy inference attack on DNN via cache-based side-channel attacks. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1515–1520. doi:10.23919/DATE54114.2022.9774742
- [30] Han Wang, Ming Tang, Quancheng Wang, Ke Xu, and Yinqian Zhang. 2025. ZenLeak: Practical Last-Level Cache Side-Channel Attacks on AMD Zen Processors. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7. doi:10.1109/DAC63849.2025.11133253

- [31] Wenjie Xiong and Jakub Szefer. 2020. Leaking information through cache LRU states. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 139–152. doi:10.1109/HPCA47549.2020.00021
- [32] Tianhong Xu, Aidong Adam Ding, and Yunsu Fei. 2025. EXAM: Exploiting Exclusive System-Level Cache in Apple M-Series SoCs for Enhanced Cache Occupancy Attacks. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*. 1294–1308. doi:10.1145/3708821.3710844
- [33] Mengjia Yan, Christopher W Fletcher, and Josep Torrellas. 2020. Cache telepathy: Leveraging shared resource attacks to learn {DNN} architectures. In *29th USENIX Security Symposium (USENIX Security 20)*. 2003–2020. <https://www.usenix.org/conference/usenixsecurity20/presentation/yan>
- [34] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*. PMLR, 40–48. <http://proceedings.mlr.press/v48/yanga16.html>
- [35] Jiyong Yu, Aishani Dutta, Trent Jaeger, David Kohlbrenner, and Christopher W Fletcher. 2023. Synchronization Storage Channels (S2C): Timer-less Cache Side-Channel Attacks on the Apple M1 via Hardware Synchronization Instructions. In *32nd USENIX Security Symposium (USENIX Security 23)*. 1973–1990. <https://www.usenix.org/conference/usenixsecurity23/presentation/yy-jiyong>
- [36] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385* (2024). doi:10.48550/arXiv.2401.02385
- [37] Zhenkai Zhang, Kunbei Cai, Yanan Guo, Fan Yao, and Xing Gao. 2024. {Invalidate+ Compare}: A {Timer-Free}{GPU} Cache Attack Primitive. In *33rd USENIX Security Symposium (USENIX Security 24)*. 2101–2118. <https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-zhenkai>
- [38] Yuanxin Zhuang, Chuan Shi, Mengmei Zhang, Jinghui Chen, Lingjuan Lyu, Pan Zhou, and Lichao Sun. 2024. Unveiling the secrets without data: Can graph neural networks be exploited through {Data-Free} model extraction attacks?. In *33rd USENIX Security Symposium (USENIX Security 24)*. 5251–5268. <https://www.usenix.org/conference/usenixsecurity24/presentation/zhuang>

## Appendix

### A Open Science

In line with the CCS open-science policy, our artifact is publicly available at [https://github.com/xth7z/SLAC\\_CCS2026](https://github.com/xth7z/SLAC_CCS2026). It contains (1) the implementation of C<sub>PRIME</sub>+C<sub>PROBE</sub> and G<sub>PRIME</sub>+C<sub>PROBE</sub> (Section 4), (2) the trace-processing scripts of Section 5.1 that reproduce Figures 7 and 8, and (3) the GNN edge recovery (Section 5.2) and LLM input/output recovery (Section 5.3) attack code. Following the included instructions, evaluators can reproduce all major experiments reported in the paper.

### B Ethical Considerations

All experiments were conducted on hardware owned by the authors, using victim workloads that we ran ourselves, so no third-party users, services, or production systems were profiled. The GNN evaluations use standard public benchmarks, and the LLM evaluations use the public MEDQUAD and FINANCIALQA prompt datasets, which contain no personally identifiable information; the study thus involved no human subjects. We believe publishing the reverse-engineering methodology, the side-channel techniques, and the end-to-end attacks is necessary for the community to mitigate this class of leakage, and we release our tools so that others can verify the vulnerability and build defenses. As part of responsible disclosure, we have reported this vulnerability to Apple’s Product Security team and are coordinating with Apple on remediation.

### C SLC copy verification protocol

In this section, we investigate whether the SLC creates an extra copy when a CPU-owned block becomes shared with the GPU, using CPU-side latency measurement. We compare two sequences of accessing two buffers (each of the size of SLC) that differ only

in whether the CPU first accesses a buffer (therefore warms up its local caches):

---

#### Algorithm 1 CPU warms target, then GPU shares it

---

- 1: Allocate two disjoint buffers: buffer1 (target) and buffer2 (probe), each about the SLC size.
  - 2: CPU sequentially accesses buffer1 to keep it in CPU private caches.
  - 3: GPU accesses buffer2 to populate the SLC with buffer2.
  - 4: GPU accesses buffer1 so that the block becomes shared.
  - 5: CPU accesses buffer2 and records the latency.
- 

*Sequence A: CPU warmed target.* If sharing buffer1 created an extra SLC copy, some lines of buffer2 would be evicted, and the CPU probe would look like a memory access (Algorithm 1). We instead observe SLC hit latencies. This means no new SLC copy is created when the target is already present in CPU private caches; the GPU obtains the data via the coherence path, and the CPU lines remain resident.

---

#### Algorithm 2 GPU brings the target into the SLC, then CPU probes

---

- 1: Allocate buffer1 and buffer2 as above.
  - 2: GPU accesses buffer2 to populate the SLC with buffer2.
  - 3: GPU accesses buffer1 without prior CPU warmup.
  - 4: CPU accesses buffer2 and records the latency.
- 

*Sequence B: GPU first touch of target.* Here, the first touch to buffer1 must insert it into the SLC (Algorithm 2). The CPU probe to buffer2 shows memory level latency, which is consistent with eviction of buffer2 lines to make room for buffer1.

In the first sequence, the CPU loads buffer1 into private caches, and the GPU loads buffer2 to SLC. The GPU then accesses buffer1, and the CPU finally probes buffer2 with time measurement. The latency remains at SLC hit latency, indicating that when buffer1 becomes shared from CPU-private, no second copy is created in SLC and the GPU obtains the data directly through the coherence protocol, an unusual *non-inclusive behavior* on the GPU side. In the second sequence, we omit the CPU warmup of local caches with buffer1, so the GPU loads buffer2 into SLC first, followed by buffer1. When CPU probes buffer2, all accesses hit the memory, indicating that buffer1 indeed enters the SLC and has evicted prior contents.

### D Single-Node Edge Recovery

This appendix details the single-node edge recovery procedure referenced in Section 5.2. Throughout, we use the following notation:

- $m$ : the true number of neighbors of the target node  $n$ ;
- $C$ : the ground-truth neighbor set, with  $|C| = m$ ;
- $\hat{C}$ : the intermediate candidate set produced by the filtering step;
- $\hat{m}$ : the neighbor count estimated from the access vector  $\mathbf{v}$ ;
- $\tilde{C}$ : the final returned set after pruning.

The recovery proceeds in three steps: (1) filtering all nodes to obtain  $\hat{C}$ , (2) estimating  $\hat{m}$  and assessing its reliability via a dataset-specific trustworthiness threshold  $T$ , and (3) pruning  $\hat{C}$  to produce  $\tilde{C}$ .

### Step 1: Filtering

A node  $i$  is admitted to  $\hat{C}$  when its profiled cache pattern  $\mathcal{M}_G[i]$  is element-wise contained in the observed access vector  $\mathbf{v}$ :

$$\hat{C} = \{i \in \{1, \dots, N\} : \mathbf{v}[j] \geq \mathcal{M}_G[i, j], \forall j \in \{1, \dots, 4096\}\}. \quad (3)$$

Every true neighbor's footprint must appear in  $\mathbf{v}$ , so this step preserves all true neighbors, i.e.,  $C \subseteq \hat{C}$ . However,  $\hat{C}$  may also contain unrelated nodes whose individual footprints happen to fall within the combined footprint of all true neighbors, inflating  $\hat{C}$  relative to  $C$ . The subsequent steps use the estimated neighbor count  $\hat{m}$  to remove these spurious candidates whenever doing so is reliable.

### Step 2: Estimating $\hat{m}$ and Deriving the Trustworthiness Threshold $T$

The neighbor count is estimated by dividing the total observed cache-line occupancy by the average per-node footprint:

$$\hat{m} = \frac{1}{\bar{f}} \sum_{j=1}^{4096} \mathbf{v}[j], \quad (4)$$

where  $\bar{f}$  is the average number of cache lines occupied by one node's feature vector in the dataset (column #Lines in Table 2). The estimator  $\hat{m}$  deviates from the true  $m$  due to two error sources.

*Discretization error.* All node feature vectors share the same dimension, but their starting offsets within cache lines vary, so each node occupies either  $\lfloor \bar{f} \rfloor$  or  $\lceil \bar{f} \rceil$  cache lines. Let  $p = \bar{f} - \lfloor \bar{f} \rfloor$  denote the probability that a node occupies one extra line. Modeling the per-node extra-line indicator as i.i.d. Bernoulli, the total line count across  $m$  neighbors has variance  $m p(1 - p)$ , giving a standard deviation of

$$\sigma_{\text{disc}}(m) = \sqrt{m p(1 - p)}. \quad (5)$$

*Side-channel noise.* The second source of error stems from side-channel measurement inaccuracies. We empirically model this as a relative error coefficient  $\eta$  applied to the total cache-line occupancy, so that the induced absolute error in  $\sum_j \mathbf{v}[j]$  scales with  $m\bar{f}$ :

$$\sigma_{\text{noise}}(m) = \eta \cdot m \cdot \bar{f}. \quad (6)$$

From our experimental measurements, we estimate  $\eta = 0.6\%$  for CPRIME+CPROBE and  $\eta = 0.4\%$  for GPRIME+CPROBE, reflecting the cleaner SLC state produced by GPU-side priming.

Dividing both error terms by  $\bar{f}$  converts them into deviations on  $\hat{m}$ :

$$\Delta_{\text{disc}}(m) = \frac{\sqrt{m p(1 - p)}}{\bar{f}}, \quad \Delta_{\text{noise}}(m) = \eta \cdot m. \quad (7)$$

For  $\hat{m}$  to round to the correct integer with at least 95% confidence, the combined deviation must stay below 0.5, where the (random) discretization term is scaled by the 95% normal quantile 1.96:

$$1.96 \cdot \Delta_{\text{disc}}(m) + \Delta_{\text{noise}}(m) < 0.5. \quad (8)$$

We define the *trustworthiness threshold*  $T$  as the largest  $m$  at which (8) still holds, i.e.,  $T$  is the positive solution to

$$1.96 \cdot \frac{\sqrt{T p(1 - p)}}{\bar{f}} + \eta \cdot T = 0.5 \quad (9)$$

which is a quadratic equation in  $\sqrt{T}$  and admits a closed-form solution. The resulting  $T$  is dataset- and configuration-specific: a larger

$\bar{f}$  or a smaller  $p$  (more uniform per-node line counts) increases  $T$ , whereas a larger  $\eta$  shrinks  $T$ . Because GPRIME drives the SLC into a cleaner state than CPRIME, GPRIME has a smaller  $\eta$  and therefore a larger  $T$  on the same dataset. The two columns labeled “Threshold  $T$ ” in Table 2 report the calibrated thresholds for each dataset under CPRIME and GPRIME.

When  $\hat{m} < T$ , the estimate is trustworthy and the pruning step (Step 3) may be safely applied. When  $\hat{m} \geq T$ , the estimate is unreliable and the procedure conservatively returns  $\hat{C}$  without pruning.

### Step 3: Pruning

Algorithm 3 consumes  $\hat{C}$ ,  $\hat{m}$ , and  $T$  and produces the final returned set  $\tilde{C}$ . The procedure first checks whether  $\hat{m}$  is trustworthy and whether pruning is necessary or computationally feasible. If pruning proceeds, it enumerates all size- $\hat{m}$  subsets of  $\hat{C}$  whose combined footprint is fully contained in  $\mathbf{v}$ , and consolidates the matching subsets by union so that any node appearing in at least one valid explanation is retained. In any non-actionable case (untrustworthy estimate, already-minimal candidate set, intractable enumeration, or no matching subset), the algorithm conservatively returns  $\hat{C}$  unchanged, preserving recall at the cost of some precision. Because

---

#### Algorithm 3 Single-Node Edge Recovery: Pruning

---

**Require:** Candidate set  $\hat{C}$ ; estimate  $\hat{m}$ ; access vector  $\mathbf{v}$ ; threshold  $T$ ; profiles  $\mathcal{M}_G$ ; enumeration cap  $K_{\text{max}} = 10^5$

**Ensure:** Returned neighbor set  $\tilde{C}$

```

1: if  $\hat{m} \geq T$  then                                ▶ Estimate untrustworthy
2:   return  $\tilde{C} \leftarrow \hat{C}$ 
3: if  $\hat{m} \geq |\hat{C}|$  then                                ▶ Candidate set already minimal
4:   return  $\tilde{C} \leftarrow \hat{C}$ 
5:  $K \leftarrow \binom{|\hat{C}|}{\hat{m}}$ 
6: if  $K > K_{\text{max}}$  then                                ▶ Enumeration intractable
7:   return  $\tilde{C} \leftarrow \hat{C}$ 
8:  $S_{\text{match}} \leftarrow \emptyset$ 
9: for all subsets  $S \subseteq \hat{C}$  with  $|S| = \hat{m}$  do
10:  if  $\mathbf{v}[j] \geq \sum_{i \in S} \mathcal{M}_G[i, j] \forall j$  then        ▶ Combined footprint
11:     $S_{\text{match}} \leftarrow S_{\text{match}} \cup \{S\}$ 
12: if  $|S_{\text{match}}| = 0$  then                                ▶ No matching subset
13:   return  $\tilde{C} \leftarrow \hat{C}$ 
14: else if  $|S_{\text{match}}| = 1$  then
15:   return  $\tilde{C} \leftarrow$  the unique  $S \in S_{\text{match}}$ 
16: else
17:   return  $\tilde{C} \leftarrow \bigcup_{S \in S_{\text{match}}} S$                 ▶ Union of matching subsets

```

---

$C \subseteq \hat{C}$  holds by construction and every fall-back branch returns  $\hat{C}$ , the procedure rarely drops true neighbors and therefore achieves near-100% recall. Precision degrades on larger or denser graphs, where the combined footprint of many neighbors saturates the SLC, inflates  $\hat{C}$ , and either pushes  $\hat{m}$  above  $T$  or makes the size- $\hat{m}$  enumeration intractable—both of which trigger the conservative fall-back to  $\hat{C}$ .